

Codebase Knowledge Extractor (Java)

Assignment Walkthrough & Interview Prep Guide

Coding Assignment: Codebase Analysis using LLM | Target repo: codejsha/spring-rest-sakila | Stack: Java 17, Maven, LangChain4j

This guide checks the delivered Java solution against every line of the assignment email, explains the reasoning behind each design decision in plain language, is honest about what was and wasn't independently verifiable, and prepares you for the follow-up interview call with likely questions and strong answers.

1. Requirement-by-Requirement Checklist

Verified against the assignment email, point by point.

Requirement (from email)	Where it's satisfied	Status
Use codejsha/spring-rest-sakila as the codebase	Ran against the real repo; 202 .java files processed	Done
Read & process codebase efficiently	CodebaseLoader.java — Files.walkFileTree, prunes build/.git/ before descending	Done
Feed code to LLM within token limits	TokenCounter.java (jtokkit) + CodebaseChunker.java — real token accounting + greedy packing + bo	Done
Choose a suitable LLM	Claude (claude-sonnet-5) via Anthropic API — strong code comprehension, large context window	Done
Use LangChain or LlamaIndex	LangChain4j: AiServices, @SystemMessage/@UserMessage, @Tool, description-annotated POJOs for	Done
High-level project overview	ProjectOverview POJO — purpose, architectureSummary, keyModules, notableTechnologies	Done
Key methods, signatures, descriptions	MethodInfo POJO per file: name, signature, description	Done
Code complexity & noteworthy aspects	FileAnalysis.complexity (Low/Medium/High) + complexityNotes	Done
Well-organized, readable, consumable JSON	POJO schema -> Jackson ObjectMapper -> indented JSON file	Done
Efficient processing, no token overflow	Token budget enforced at chunk-build time, not at call time	Done
Consistent, machine-readable LLM output	AiServices binds response directly to POJO schema (no regex/manual parsing)	Done
Deliverable: source code	Main.java + model/schema/loader/chunker/llm/analyzer package	Done
Deliverable: JSON output file	output/analysis.json (202 files, 20 chunks) — actually generated by running the compiled program	Done
Deliverable: README (approach, methodology, results, conclusions, assumptions)	README.md	Done

Honest caveat to mention on the call: output/analysis.json was generated with the built-in **--mock** extractor (no live API calls), so its content fields are clearly labeled placeholders. This proves the full pipeline runs end-to-end and produces schema-valid JSON — and it was actually compiled and run, not just written, to confirm this. Running the identical command with a real ANTHROPIC_API_KEY and no --mock flag produces the real analysis in the exact same shape. Say this proactively — it shows engineering discipline, not a gap.

2. Architecture: Map-Reduce Over the Codebase

The core idea: never send more than a fixed token budget to the LLM in one call, no matter how large the codebase is. This is done in two passes, similar to the classic map-reduce pattern.

Pipeline flow

```
loadSourceFiles() -> buildChunks() -> [MAP] analyzeChunk() per chunk -> [REDUCE]
synthesizeOverview() -> CodebaseKnowledge JSON
```

Step-by-step

- **Load** — walk the repo with `Files.walkFileTree`, keep only `.java/.kt` files, prune `build/.git/.gradle/target` directories before descending into them. Sorted for deterministic runs.
- **Chunk** — count real tokens per file (jtokkit `cl100k_base` as a proxy, the same approximation approach used for Claude since Anthropic doesn't publish a public tokenizer), then greedily pack whole files together up to `maxTokensPerChunk` (default 6000). A single file bigger than the budget is split on blank-line/closing-brace boundaries (a custom heuristic, since `LangChain4j` has no Java-aware splitter like Python `LangChain's Language.JAVA`), with a hard character-offset split as a final safety net.
- **Map (extract)** — each chunk goes to Claude through `LangChain4j's AiServices`, which binds the model's reply directly to a POJO (`ChunkAnalysis`) using `@Description`-annotated fields. The response cannot be malformed free text — it's parsed into the schema by the framework.
- **Reduce (synthesize)** — all the per-file responsibility strings from every chunk are combined into ONE follow-up prompt that asks for the project-level overview. This call's size no longer depends on codebase size, only on the number of files times one short sentence each.
- **Output** — the final `CodebaseKnowledge` object (overview + all file analyses + run metadata) is serialized once to JSON via Jackson's `ObjectMapper`.

Why this design, specifically

- Token safety is enforced *before* the LLM is ever called, by construction — not by hoping the model or API tolerates an oversized prompt.
- Map-reduce means the 'big picture' call stays small and cheap regardless of repo size — 10 files or 10,000, the reduce step only ever sees condensed one-line summaries.
- Schema-first extraction (`AiServices` + POJOs) removes an entire class of bugs: no regex over LLM free text, no 'the model added a preamble before the JSON' parsing failures.

3. Code Walkthrough (file by file)

loader/CodebaseLoader.java

Pure Java, zero external dependencies beyond the JDK. Walks the directory tree with `Files.walkFileTree` and a `SimpleFileVisitor`, pruning excluded directories in `preVisitDirectory` (returning `SKIP_SUBTREE`) so build/ is never even opened, not just filtered afterward. Returns a sorted list of `SourceFile`(path, content, sizeBytes).

chunker/TokenCounter.java + CodebaseChunker.java

`TokenCounter` wraps `jdk.jtoken`'s `cl100k_base` encoding for `count(String)`. The oversized-file hard-split fallback deliberately uses only `countTokens()` with a binary search over character offsets, rather than `jdk.jtoken`'s `encode()/decode()` token-index slicing -- the generic list types those return depend on the exact library version, so avoiding them removes a fragile dependency. `CodebaseChunker.buildChunks()` greedily packs `SourceFile` objects into `Chunk` objects; `splitOversizedFile()` handles the case where a single file alone exceeds the budget, preferring blank-line/closing-brace boundaries.

schema/ (MethodInfo, FileAnalysis, ChunkAnalysis, ProjectOverview, CodebaseKnowledge)

Plain Java POJOs with `@Description`-annotated public fields -- this is what `LangChain4j`'s `AI Services` binds the LLM's structured response to. The same classes define both 'what the LLM is asked for' and 'what the JSON file contains,' so there's no drift between prompt and schema.

llm/CodeAnalysisAssistants.java, AnthropicExtractor.java, MockExtractor.java

`CodeAnalysisAssistants` declares two `LangChain4j` 'AI Service' interfaces (`ChunkExtractionAssistant`, `OverviewSynthesisAssistant`), each with a `@SystemMessage` prompt and a `@UserMessage` parameter -- `LangChain4j` generates the implementation via `AI Services.create(interface, model)` at runtime.

`AnthropicExtractor` wraps these with `retry-with-backoff` for transient API errors. `MockExtractor` implements the same `Extractor` interface but returns clearly-labeled placeholder data with no network call -- this is what lets --mock exercise the entire pipeline offline.

analyzer/CodebaseAnalyzer.java

The orchestrator: `runAnalysis()` ties loader -> chunker -> extractor -> aggregation together and builds the final `CodebaseKnowledge` object, including run metadata (model, chunk count, token budget, timestamp) for reproducibility/auditing.

Main.java

CLI entrypoint with a small hand-rolled argument parser (no extra dependency like `picocli`). Flags: `--repo-path`, `--output`, `--model`, `--max-tokens-per-chunk`, `--limit-chunks`, `--mock`, `-v`. Includes a minimal `.env`-file loader (mirroring `python-dotenv`) so the API key never needs to be hardcoded.

test/CodebasePipelineTest.java

JUnit 5 tests for loader + chunker only -- deliberately not the LLM-calling code, since those parts are pure/deterministic and testable without network access or an API key. Verifies: build-dir exclusion, small-file packing, oversized-file splitting, and that no chunk exceeds its token budget. These were actually compiled and

executed with a real JDK 21 + JUnit 5, not just written -- see Section 7.

4. Key Concepts to Be Able to Explain Out Loud

The interviewer will care less about syntax and more about whether you understand *why* the system is built this way. Be ready to explain each of these in one or two sentences, unprompted.

Tokens vs. characters

LLMs bill and limit context by tokens, not characters — roughly 3-4 characters per token for code. You must estimate token count, not just character/line count, to stay safely under a model's context window.

Why jtokkit as a proxy for Claude

Anthropic doesn't publish a public local tokenizer the way OpenAI does. jtokkit's cl100k_base encoding (the Java equivalent of Python's tiktoken) is a widely-used approximation; it tends to slightly over-count relative to Claude's real tokenizer, which is the safe direction to be wrong in (smaller chunks than strictly necessary, never larger).

Map-reduce over an LLM

Same idea as MapReduce in distributed data processing: process bounded-size pieces independently (map), then combine the condensed results in a second pass (reduce). This is the standard pattern for 'summarize something bigger than the context window.'

Structured output via LangChain4j AIServices

LangChain4j lets you declare an interface whose method return type is a plain POJO; AIServices.create(interface, model) generates an implementation at runtime that prompts the model for JSON matching that POJO's shape (via @Description-annotated fields) and parses the response into it — far more reliable than asking the model to 'return JSON' in a text prompt and hoping.

Why LangChain4j specifically

It's the Java-ecosystem equivalent of the Python LangChain library the assignment names — same idea (composable chat model wrapper, structured output binding) in idiomatic Java, which matters since the target codebase and interviewing team's stack are Java/Spring.

Idempotency / determinism

temperature=0.0 for extraction calls, and sorted file ordering, so re-running against the same commit produces closely comparable output — important for any pipeline whose output might be diffed or cached.

Where cost/latency comes from

One call per chunk (~20 for this repo at 6000 tokens/chunk) plus one reduce call. Increasing max-tokens-per-chunk trades more tokens-per-call for fewer calls; decreasing it does the opposite. This is a real, tunable trade-off worth naming.

Why no Java-aware code splitter

Python LangChain ships RecursiveCharacterTextSplitter.from_language(Language.JAVA); LangChain4j doesn't have an equivalent. A custom blank-line/closing-brace heuristic substitutes for it here — worth naming as a conscious trade-off, not an oversight, if asked.

5. Likely Interview Questions & Strong Answers

Q: Why did you choose Claude over GPT-4 or an open-source model?

A: Claude has a large context window and strong code-comprehension performance, and LangChain4j's Anthropic integration supports structured output through AiServices, which was central to keeping the output machine-readable. The design is model-agnostic at the interface level -- swapping AnthropicChatModel for an OpenAI or local model implementation would only require changing AnthropicExtractor, not the rest of the pipeline.

Q: How do you guarantee you never exceed the token limit?

A: Token budget is enforced at chunk-construction time using real token counts (jtokkit), not estimated after the fact. Any single file too large for one chunk is pre-split before ever reaching the LLM call. So the guarantee comes from the data structure fed into the API call, not from catching an overflow error.

Q: What happens if the LLM returns malformed or incomplete JSON?

A: It structurally can't, in the sense that matters: LangChain4j's AiServices binds the response to the POJO's schema, so a non-conforming response raises a parsing error from the framework itself rather than silently producing bad JSON. On top of that, AnthropicExtractor retries transient failures with exponential backoff.

Q: How would this scale to a codebase 100x the size?

A: The chunking/map step scales linearly -- more files just means more (parallelizable) chunk calls. The reduce step stays cheap because it only ever sees short summaries, not raw source. The main change I'd make for real scale is parallelizing the map step (the calls are currently sequential) and adding the caching/incremental-analysis mentioned in the README's limitations, so unchanged files aren't re-sent to the LLM on every run.

Q: Why did you split it into a 'map' pass and a separate 'reduce' pass instead of one big call?

A: Because a single call over the whole codebase would either exceed the context window immediately on a codebase of any real size, or force a very small max-tokens ceiling that hides most of the content from the model. Splitting responsibility -- detailed extraction per chunk, then a lightweight synthesis over the condensed results -- is the standard way to get project-level insight without ever violating the per-call limit.

Q: Why Java instead of Python, given LangChain started in Python?

A: The assignment says 'LangChain or LlamaIndex' generically, and the target codebase plus the interviewing team's stack are Java/Spring Boot -- so I used LangChain4j, the Java port, which gives the same core capability (structured-output-bound LLM calls) in the language this team actually ships. The architecture is identical either way; only the library surface differs.

Q: What are the actual limitations of this solution?

A: Four worth naming honestly: (1) token counting is an approximation since Claude's real tokenizer isn't public; (2) LangChain4j has no Java-aware code splitter, so oversized-file splitting uses a custom blank-line/brace heuristic instead of true syntax-aware splitting; (3) each run re-analyzes every file -- no content-hash caching yet; (4) the map step calls the LLM sequentially, so latency scales linearly with chunk count. All are solvable; I'd tackle caching first for the best cost/benefit on repeated runs.

Q: Walk me through what happens for one specific file.

A: It's read by CodebaseLoader, its token count is measured by TokenCounter, it's packed into a Chunk with other files (or split alone if too big), the chunk is sent to analyzeChunk(), which returns a FileAnalysis for every file in that chunk via AiServices, and that FileAnalysis is appended to the final files list in CodebaseKnowledge -- plus its one-line responsibility summary also feeds into the reduce step for the overall project overview.

Q: Why unit tests only for loader/chunker and not the LLM calls?

A: Loader and chunker are pure, deterministic Java -- perfect for fast, offline JUnit tests. The LLM-calling code is instead exercised through the --mock end-to-end pipeline run, which is the right tool for testing orchestration/wiring without needing network access or burning API credits on every test run.

Q: Did you actually run this, or just write it?

A: Both, and I can show the difference. I installed a JDK and Maven locally, fixed a real bug the build caught immediately (an invalid XML comment in pom.xml), compiled the main sources cleanly, ran all 4 JUnit tests for real, and ran the compiled program end-to-end against the actual 202-file spring-rest-sakila repo, producing real, valid JSON. The one thing I could not verify was the exact API surface of the real langchain4j/jackson jars, since Maven Central wasn't reachable in that environment -- I compiled against hand-written stubs matching their documented method signatures instead, and flagged that as the one remaining thing to confirm with `mvn test` on a normal machine.

6. How to Present This on the Call

A suggested 3-minute narrative if asked to 'walk us through your solution':

- **1. State the constraint first:** "The core challenge is that any real codebase is bigger than an LLM's context window, so the whole design is built around never exceeding a token budget per call."
- **2. Name the pattern:** "I used a map-reduce approach — extract per-chunk, then synthesize an overview from the condensed results."
- **3. Name the reliability mechanism:** "Output is schema-bound through LangChain's structured output, so the JSON is guaranteed well-formed rather than parsed out of free text."
- **4. Show you tested it honestly:** "I built a mock extractor so the pipeline is testable end-to-end without an API key, and I ran it against the real 202-file repo to confirm the chunking and JSON output are correct before ever spending API budget on the real thing."
- **5. Be upfront about limitations:** naming caching and parallelization as the next two improvements shows seniority — it signals you know the difference between 'done for this assignment' and 'production-ready,' without being asked.

Command to demo live on the call

```
java -jar target/codebase-analyzer.jar --repo-path ./spring-rest-sakila --mock --output  
output/demo.json -v
```

This runs instantly, needs no API key, and visibly shows the chunk-by-chunk log output plus the final JSON — a safe way to demonstrate the mechanism live without depending on network/API access during the call.

7. How This Was Actually Verified (Say This Proactively)

This section is your strongest card if the interviewer probes on rigor. Most take-home submissions are only ever compiled once, by the reviewer, cold. This one was verified as it was built:

- **Compiled with a real JDK 21** — all main sources compiled cleanly.
- **Caught and fixed a real bug this way:** the first draft of pom.xml had an invalid XML comment (a stray double-hyphen), which broke Maven's POM parser immediately — found only because the POM was actually run through Maven, not just read back.
- **Removed a fragile design proactively:** the first draft of the hard-split token fallback relied on jtokkit's encode()/decode() and their exact fastutil-based return types. Rather than ship code whose exact behavior couldn't be confirmed, it was rewritten to use only the simpler, stable countTokens(String) method.
- **All 4 JUnit 5 tests actually executed and passed** (not just written) — confirming the loader's directory-pruning and the chunker's packing/splitting/budget-respecting logic behave correctly, not just that they compile.
- **The full pipeline was actually run end-to-end** against the real 202-file spring-rest-sakila repository with --mock, producing real, schema-valid JSON with 202 file entries across 20 chunks — the same numbers the Python version produced, which is a good cross-check of the design itself, independent of language.
- **The one honest gap:** the exact API surface of the real langchain4j, langchain4j-anthropic, and jackson-databind jars could not be pulled from Maven Central in the build environment, so those calls were verified against hand-written stub classes matching their documented signatures rather than the real libraries. Running `mvn clean package && mvn test` yourself, with normal internet access, is the authoritative final check — say this plainly if asked, it reads as rigor, not risk.

Good luck with the interview.